The ART of LLM Refinement: Ask, Refine, and Trust

Kumar Shridhar [◇] [*] **Koustuv Sinha** [♠] **Andrew Cohen** [♠] **Tianlu Wang** [♠] **Ping Yu** [♠]
Ram Pasunuru [♠] **Mrinmaya Sachan** [◇] **Jason Weston** [♠] **Asli Celikyilmaz** [♠]

[◇] ETH Zurich [♠] Meta AI

arXiv:2311.07961v1 [cs.CL] 14 Nov 2023

Abstract

Large Language Models (LLMs) have demonstrated remarkable generative abilities, but can they judge the quality of their own generations? A popular concept, referred to as *self-refinement*, postulates that LLMs can detect and correct the errors in their generations when asked to do so. However, recent empirical evidence points in the opposite direction, suggesting that LLMs often struggle to accurately identify errors when reasoning is involved. To address this, we propose a reasoning with refinement strategy called ART: Ask, Refine, and Trust, which *asks* necessary questions to decide when an LLM should *refine* its output, and either affirm or withhold *trust* in its refinement by ranking the refinement and the initial prediction. On two multistep reasoning tasks of mathematical word problems (GSM8K) and question answering (StrategyQA), ART achieves a performance gain of 5 points over self-refinement baselines, while using a much smaller model as the decision maker. We also demonstrate the benefit of using smaller models to make refinement decisions as a cost-effective alternative to fine-tuning a larger model.

1 Introduction

The ability of Large Language Models (LLMs) to generate coherent and meaningful text has improved significantly over the years (OpenAI, 2023). However, LLMs often exhibit inaccuracies in their initial generations, and it has been posited that iterative refinement can rectify their errors (Madaan et al.; Shridhar et al., 2023a; Welleck et al.; Zheng et al., 2023). Madaan et al. demonstrated the potential of *self-refinement* for diverse tasks such as dialogue response and sentiment reversal; however, this approach proved less effective when applied to mathematical reasoning. Similarly, Shridhar et al. (2023a) and Huang et al. (2023) further demonstrated the challenges LLMs face in identifying errors in reasoning tasks. Developing models that consistently evaluate and correct their errors would be a valuable step towards building more reliable language models.

Through empirical observation on two multistep reasoning datasets, we find that *self-refinement* does not reliably improve initial generations, validating the previous findings of Huang et al. (2023). In fact, in the majority of cases, *self-refinement* has a detrimental effect on performance. On the other hand, fine-tuning language models usually improves their performance on a given task by facilitating better adaptation to the task objectives (Yuan et al., 2023). Smaller models can be trained on LLMs' data to improve their performance, which can serve as cost-effective alternatives to LLMs for the given task (Magister et al., 2023; Shridhar et al., 2023b; Hsieh et al., 2023). This led us to explore the possibility of training a smaller model as a decision maker for refinement, which can consistently determine *when to refine*, while the larger model can subsequently perform the refinement process.

In our work, we propose a refinement approach called ART: Ask, Refine, and Trust, which, given an initial LLM response, works in the following three stages: (a) evaluating whether the initial generation requires refinement by asking a series of questions (Ask); (b) executing the *refinement* step based on the evaluation (Refine); and finally (c) selecting either the refined result or the initial prediction (Trust). On two multistep reasoning tasks, mathematical reasoning and question answering, we illustrate the effectiveness of ART by training models of different sizes. We observe that a much smaller model (LLaMA 7B; Touvron et al., 2023) trained to decide *when to refine*, can outperform a 10x larger model (LLaMA 70B) in a *self-refinement* setup (by up to 5 points). In addition, we evaluate the cost and accuracy tradeoffs of

[*] Work done during internship at Meta AI; correspondence at shkumar@ethz.ch

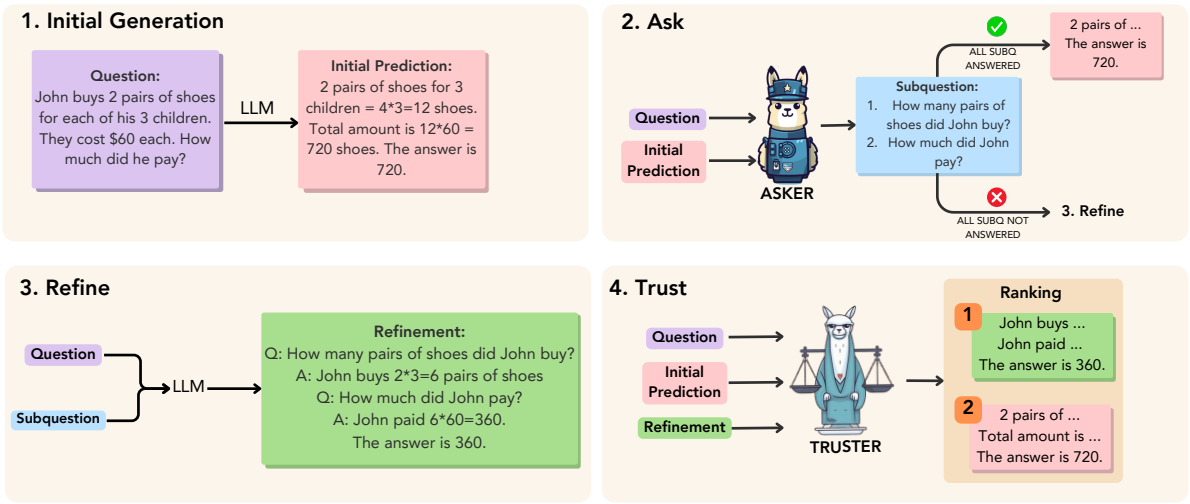

Figure 1: Our proposed objective: ART: Ask, Refine, and Trust during inference. Given a problem, an LLM first generates an initial prediction which is sent to an Asker that asks relevant questions (sub-questions) to decide whether refinement is needed or not. If all sub-questions are answered, it returns the initial prediction and no refinement is needed. If not, the model refines the initial prediction using the subquestions. Finally, the initial prediction and the refined response is sent to the Truster, which ranks them to decide if refinement was needed or if the initial prediction was better.

training a smaller model with ART to make a refinement decision for a pretrained LLM vs fine-tuning the LLM. In many cases, we illustrate the cost-effectiveness of ART as a viable alternative to fine-tuning LLMs. Finally, we show that our trained models (Asker and Truster) can work seamlessly across a wide range of LLMs (LLaMA 70B (Touvron et al., 2023), ChatGPT (Brown et al., 2020) and GPT-4 (OpenAI, 2023)) without requiring additional modifications.

2 Related Work

Strategies that use intermediate computation to solve reasoning tasks such as chain of thought (Wei et al., 2022; Lewkowycz et al., 2022; Zhang et al., 2022; Kojima et al., 2022; Wang et al., 2022a; LYU et al., 2023) and subquestion decomposition (Min et al., 2019; Shridhar et al., 2022; Zhou et al., 2023; Radhakrishnan et al., 2023) have proven to be very effective. Most LLM refinement techniques use one of these two strategies (Madaan et al.; Welleck et al.; Huang et al., 2023; Paul et al., 2023; Yoran et al., 2023) or occasionally a combination of the two (Shridhar et al., 2023a). Shridhar et al. (2023a) unified past *reasoning with refinement* methods under a common umbrella of *sampling* (given a query, LLM generates the initial response), *re-sampling* (LLM refines the initial response), and *selection* (choose either the refinement or rollback to initial response). However, a single LLM was used to perform the initial generation, refinement and later selection by using different prompts. We, on the other hand, propose to train a separate, much smaller capacity model to make refinement decisions and later decide whether or not to trust the refinement over the initial generation and are not limited to prompting-based solutions.

Asking questions to verify facts present in the model prediction has been studied by Dhuliawala et al. (2023) in the context of hallucination detection. However, this work only deals with hallucinations in the form of directly stated factual inaccuracies. It is important to note that hallucinations can take many other forms, including incorrect reasoning steps. To address this, we train an expert model to verify each reasoning step by asking relevant questions.

Training a model to rank the outputs has been studied in the past in various contexts (Burges et al., 2005), including but not limited to text generation (Krishna et al., 2022), mathematical reasoning problems (Cobbe et al., 2021), machine translation (Tunstall et al., 2023), and so on. However, we do not study the standard setting of training to rank the quality of generations, but rather to decide if the refinement led to incorrect results and if it needs to be rolled back. This has some additional similarities to rejection sampling fine-tuning (Yuan et al.,

2023), where a model is trained to generate and collect the correct reasoning chains as augmented fine-tuning datasets. On the other hand, we collect both correct and incorrect reasoning chains for ranking the outputs.

Finally, our work is similar to distilling reasoning skills into smaller models (Shridhar et al., 2023b; Magister et al., 2023; Hsieh et al., 2023). However, instead of teaching smaller models to reason, we train smaller models to ask questions to verify the reasoning and decide whether the reasoning is correct, which differs from asking questions as planning to reason (Shridhar et al., 2022).

3 ART: Ask, Refine, and Trust

In this section, we define the objective of our proposed methodology ART: Ask, Refine, and Trust in detail. Given a query and an initial prediction generated by the LLM, ART uses a trainable pipeline for refinement as follows: (a) evaluate whether the initial generation requires refinement by asking a series of questions (Ask); (b) perform the *refinement* step based on the evaluation (Refine); and finally (c) choose either the refined result or the initial prediction (Trust).

3.1 Initial Prediction

Given a task query x, the LLM ψ generates an initial prediction $\mathsf{y} = \psi(\mathsf{x})$. For pre-trained LLMs, the query x is augmented with several examples of the task as few-shot prompts, while for fine-tuned models the query is provided directly without any examples. Due to the multi-step reasoning nature of the tasks where intermediate steps are beneficial for the model to arrive at the final answer, we consider using Chain of Thought (CoT; Wei et al., 2022) and Subquestion Decomposition (Decomp; Shridhar et al., 2022; Zhou et al., 2023) as two of our main methods for initial prediction.

3.2 Ask

Once the initial prediction is generated, the next step is to decide when to refine the output. Refining every sample often leads to much worse performance (Huang et al., 2023). Thus, we train an Asker to determine whether a prediction is correct or not, and then refine only the samples about which the Asker is uncertain about. However, before a smaller model can determine whether a generated answer is correct or whether refinement is needed, it is important to align the model with task-specific knowledge and the expected outcome. We fine-tune the smaller model in CoT style (intermediate steps with the final answer, as shown by the "Initial Prediction" in Figure 1) on the training data. Next, we create the dataset for training the Asker model. We use the LLM ψ to generate k predictions per example on the training set, and then label them "Yes" or "No" for refinement based on whether the prediction was correct or incorrect (exact numbers are presented in Table 1). For each prediction, we append the subquestions present in the datasets [1] prior to the "Yes" or "No" decision to further train the fine-tuned model. In this way the Asker learns to first ask the relevant questions, map them to the prediction and then decide whether all its questions are answered in the prediction or not, leading to the refinement decision. An example is presented in the appendix Figure 6.

Dataset	Train Samples		
	Fine-tune	**Asker**	**Truster**
GSM8K	7473	35000	15000
StrategyQA	1832	9000	2300

Table 1: Comparison of different data sizes used for fine-tuning, and training the Asker and Truster models.

3.3 Refine

If the Asker predicts "Yes" (refinement is needed), then the LLM ψ is used to refine the output given the input and the subquestions from the Asker model, $\mathsf{y}_{\text{ref}} = \psi(\mathsf{x};\mathsf{subq})$. Similar to Shridhar et al. (2023a), for the StrategyQA dataset, additional facts (facts) are also provided to the model ψ during refinement ($\mathsf{y}_{\text{ref}} = \psi(\mathsf{x};\mathsf{subq};\mathsf{facts})$). An example is presented in appendix Figure 7.

3.4 Trust

Finally, to decide whether the refinement output should be preferred over the original generation, we train a Truster that takes two candidates (y, y_{ref}) for the task query x and decides which one to prefer over the other. An example is presented in the appendix section 7. However, in 80% of the cases, the final answer of the refinement y_{ref} and the initial prediction y were the same. Our goal is to make Truster learn to identify the reasoning chain with the correct final answer and not a particular styled

[1] Note that the subquestions are available for each data set and we used them to train the Asker model. However, LLMs can be used to generate the subquestions and then distill them into smaller models that perform similarly to the ground truth (Magister et al., 2023)

intermediate reasoning chain. To create a good-sized training data set, we used the same training data collected for the Asker model (Table 1) and selected the prediction samples that have both a correct and an incorrect prediction. We construct preferred (correct predictions) vs. non-preferred (incorrect predictions) pairs and train a Truster with the text classification objective as:

$$\mathcal{L}_\theta = -\mathbb{E}_{\mathsf{x}, y_j, y_k \sim \mathcal{D}} \left[\log(\sigma(r_\theta(\mathsf{x}, y_j) - r_\theta(\mathsf{x}, y_k)))\right] \quad (1)$$

where, r is the score of the Truster model, y_j is the preferred candidate (correct prediction) and y_k is the non-preferred candidate (incorrect prediction in our case) from the dataset $\mathcal{D}$. Based on the score for each sample, we select the best scored output.

4 Experiments

4.1 Dataset

We test the ART refinement strategy on two multi-step reasoning tasks, GSM8K (Cobbe et al., 2021) and StrategyQA (Geva et al., 2021). The GSM8K dataset is a grade school math word problem dataset with a training set of 7473 samples and a test set of 1319 samples, each requiring two to eight steps to solve. The dataset also consists of sub-questions that correspond to the steps in a given correct solution. StrategyQA, on the other hand, is a question-answering benchmark that focuses on open-domain questions, requiring reasoning steps to solve it. StrategyQA consists of 2290 training examples, of which the first 20% were used as the test set and the remaining 80% as the training set, following previous work (Magister et al., 2023; Shridhar et al., 2023a). Each question is accompanied by its decomposed questions and the correct factual knowledge required to answer it. Example of each dataset is presented in appendix Figure 7.

4.2 Experimental Setup

We use LLaMA 70B (pre-trained and chat) (Touvron et al., 2023), ChatGPT (turbo (gpt-3.5-turbo) and turbo-instruct (gpt-3.5-turbo-instruct)). (Brown et al., 2020), and GPT-4 (gpt-4) (OpenAI, 2023) as base models ψ due to their popularity and state-of-the-art performance. Next, we fine-tuned variants of the LLaMA model (7B, 13B, and 70B) on the GSM8K dataset and 7B and 13B on the StrategyQA dataset. All fine-tuned variants were further trained to get the Asker model to ask relevant questions and decide when to refine. Finally, we fine-tuned the LLaMA 13B model to get the Truster that decides between the original and refined output. All pre-trained and fine-tuned LLaMA models were used with greedy decoding during testing (temperature = 0 and top p = 1). To collect data for training, different temperatures were used to collect diverse samples (temperature = {0, 0.3, 0.4, 0.7, 0.8}) and k was set to 5 to generate 5 predictions on the train set. All training was done on a cluster of 8 A100s 80GB each GPUs (except for the LLaMA 70B fine-tuning, which required 4 clusters of 8 A100s each).

4.3 Results

Self-Refinement is not enough Table 2 shows the refinement framework of initial prediction, refinement, and trust. In general, the performance of LLaMA 70B is much lower than the ChatGPT turbo model for the GSM8K dataset (59 compared to 77 for CoT and 55 compared to 78 for Subquestion Decomposition). Furthermore, the Subquestion Decomposition (Decomp) approach performs better than CoT for ChatGPT, but the opposite is true for LLaMA 70B. Since the training data and the model architecture of ChatGPT are not public, it is difficult to understand the performance gap. Finally, *self-refinement* improves performance in some cases, but leads to worse performance in others (Blue colored boxes in Table 2 show the comparison). However, combining refinement with the trust module consistently improves performance over the initial prediction in almost all cases. This demonstrates the usefulness of the different components of our proposed ART methodology . Note that our baselines of the Self modules of refinement and trust uses the same prompts as presented in Shridhar et al. (2023a) for a fair comparison.

Importance of Asking Table 2 demonstrates the effectiveness of training an Asker that decides when to refine the outputs. Compared to the self-refinement (Self) strategy, a much smaller model like LLaMA 7B (Asker_{7B}) outperforms much larger LLMs like ChatGPT self-refinement (Self) by over 2 points (80.89 vs. 78.62). LLaMA 13B (Asker_{13B}) improves it by over 4 points (78.62 $\rightarrow$ 82.18). The trend is similar when refinements are compared with the self-refinement capabilities (Self) of LLaMA 70B, where a 7B model (Asker_{7B}) outperforms the pre-trained self-

Model	Initial Prediction		Refinement			Trust	
Type	**Method**	**Accuracy**	**Subquestions**	**Model**	**Accuracy**	**Model**	**Accuracy**
LLaMA 70B							
Pre-trained	CoT	59.74	No	Self	59.07	Self	59.83
Pre-trained	CoT	59.74	Yes	Self	59.83	Self	60.43
Pre-trained	Decomp	54.55	No	Self	55.11	Self	55.34
Pre-trained	Decomp	54.66	Yes	Self	50.26	Self	54.51
Pre-trained	CoT	59.74	Yes	$\texttt{Asker}_{7B}$	61.33	Truster	61.94
Pre-trained	CoT	59.74	Yes	$\texttt{Asker}_{13B}$	62.74	Truster	63.85
Pre-trained	CoT	59.74	Yes	$\texttt{Asker}_{70B}$	**63.60**	Truster	**64.24**
Chat	CoT	58.90	No	Self	59.10	Self	58.79
Chat	CoT	58.90	Yes	Self	58.83	Self	59.55
Chat	CoT	58.90	Yes	$\texttt{Asker}_{7B}$	60.12	Truster	61.18
Chat	CoT	58.90	Yes	$\texttt{Asker}_{13B}$	63.00	Truster	63.30
Chat	CoT	58.90	Yes	$\texttt{Asker}_{70B}$	**63.80**	Truster	**64.40**
ChatGPT							
Turbo	CoT^{S}	71.64	No^{S}	Self^{S}	73.00	Self^{S}	72.93
Turbo	CoT^{S}	71.64	Yes^{S}	Self^{S}	73.99	Self^{S}	73.99
Turbo	CoT^{SR}	74.58	No^{SR}	Self^{SR}	75.00	Most Recent^{SR}	75.00
Turbo	CoT^{SR}	74.58	No^{SR}	Self^{SR}	75.00	Most Recent^{SR}	75.00
Turbo	$\text{CoT}^{!C}$	75.90	$\text{No}^{!C}$	$\text{Self}^{!C}$	75.10	$\text{Most Recent}^{!C}$	75.10
Turbo	CoT	77.71	No	Self	78.16	Self	78.28
Turbo	CoT	77.71	Yes	Self	78.46	Self	78.89
Turbo	Decomp	78.62	No	Self	78.99	Self	78.99
Turbo	Decomp	78.62	Yes	Self	78.24	Self	79.22
Turbo	CoT	77.71	No	$\texttt{Asker}_{7B}$	80.89	Truster	81.14
Turbo	CoT	77.71	Yes	$\texttt{Asker}_{13B}$	**82.18**	Truster	**82.64**
Instruct	CoT	71.26	No	Self	70.28	Self	71.50
Instruct	CoT	71.26	Yes	Self	72.32	Self	72.85
Instruct	CoT	71.26	Yes	$\texttt{Asker}_{7B}$	76.19	Truster	76.34
Instruct	CoT	71.26	Yes	$\texttt{Asker}_{13B}$	**78.46**	Truster	**79.86**
GPT-4							
-	CoT^{S}	91.45	Yes^{S}	Self^{S}	90.80	Self^{S}	93.10
-	CoT^{SR}	92.90	No^{SR}	Self^{SR}	93.10	Most Recent^{SR}	93.10
-	CoT	91.88	Yes	$\texttt{Asker}_{7B}$	93.25	Truster	93.45
-	CoT	91.88	Yes	$\texttt{Asker}_{13B}$	**93.72**	Truster	**94.08**

Table 2: Accuracy (`maj1@1`) comparison between different methods and refinement strategies on the GSM8K dataset. `Initial Prediction` refers to the initial generation from the LLM with its `Method` referring to one of the reasoning strategies (Chain of Thought (CoT) or Subquestion Decomposition (Decomp) in our case). `Refinement` refers to the combination of the *Ask* and the *Refine* stages in `ART` with or without the use of subquestions during refinement (`subquestions`). Finally, `Trust` refers to the *Trust* stage in `ART`, where *Self* refers to *self-refinement*, `Truster` is our trained model and *Most Recent* refers to choosing refinement as the final result. Yellow represents the baseline methods from previous work ($(.)^{S}$ represents results from Shridhar et al. (2023a), $(.)^{SR}$ from Madaan et al., and $(.)^{!C}$ from Huang et al. (2023)), Blue represents our implementations of the baselines, and Green represents our proposed methods. Underline represents the best results from previous strategies, and **bold** represents the overall best result.

refinement capabilities of LLaMA 70B by about 2 points (61.33 vs. 59.83) and over 1 point for the chat model (58.83 vs. 60.12). The 13B model ($\texttt{Asker}_{13B}$), on the other hand, improves it by over 3 points for the pretrained LLaMA 70B model (59.83 $\rightarrow$ 62.74) and the chat version by more than 4 points (58.83 $\rightarrow$ 63.00). Finally, using the 70B model as `Asker` ($\texttt{Asker}_{70B}$) further improves the results by 4 points for the pre-trained version (59.83 $\rightarrow$ 63.60) and over 5 points for the chat version (58.83 $\rightarrow$ 63.80). The results follow a similar trend for the GPT-4 models, where both the 7B ($\texttt{Asker}_{7B}$) and 13B ($\texttt{Asker}_{13B}$) models improve the results over the initial generation by about 2 points (91.88 $\rightarrow$ 93.72), which is higher than other baselines from Madaan et al. and Shridhar et al. (2023a). Finally, note that our proposed strategy `ART` improves the overall performance of ChatGPT to 82.18 after refining with a single pass (`maj1@1`), which is similar to the self-consistency score of 3

samples (maj1@3) (Huang et al., 2023).

The results on StrategyQA follow a similar trend, where a 7B model $\texttt{Asker}_{7B}$ improves the refinement score by 1 point for LLaMA 70B (75.15 → 76.22) and over 3 points for ChatGPT (70.52 → 73.84), as shown in Table 3. Note that following Shridhar et al. (2023a), we also provide some factual information along with the questions during refinement so that the model can correct its factual inaccuracy. The gains are larger for the $\texttt{Asker}_{13B}$ model, where the performance improves by 3 points for LLaMA 70B (75.15 → 78.38) and 5 points for ChatGPT (70.52 → 75.76), demonstrating the clear importance of asking questions for refinement decision making.

Initial Pred	Refinement		Trust	
Acc	**Model**	**Acc**	**Model**	**Acc**
LLaMA 70B Pre-trained				
74.45	Self	75.15	Self	75.74
74.45	$\texttt{Asker}_{7B}$	76.22	Truster	76.12
74.45	$\texttt{Asker}_{13B}$	**78.38**	Truster	**78.44**
ChatGPT Turbo				
73.58	Self	70.52	Self	74.89
73.58	$\texttt{Asker}_{7B}$	73.84	Truster	74.04
73.58	$\texttt{Asker}_{13B}$	**75.76**	Truster	**75.86**

Table 3: Accuracy comparison on the StrategyQA dataset for refinement and trust with different models. Blue represents our implementations of the baselines, and Green represents our proposed methods.

(Don't) Always Trust Refinement Table 2 demonstrates the usefulness of a trust module that decides whether the refinement improves or degrades the initial prediction. We train a Truster model that learns to rank the initial prediction and the refined output and decides which one to choose for a given input. Our trained Truster model (LLaMA 13B) achieves an accuracy of the pre-trained LLaMA 70B of as high as 64.24, which is 4 points higher than the baseline (60.43). The trend is similar for the chat version, where the improvement is almost 5 points over the baseline method of using the same LLM for decision making (59.55 → 64.40). The results follow a similar trend for ChatGPT where the improvement over baselines (the same LLM) is about 4 points for the Turbo model over the baselines (78.89 → 82.64) and about 7 points from the best previous method of Self-Refine (Madaan et al.) (75.10 of Self-Refine → 82.64). The gains for GPT-4 are very small, possibly due to the high performance of the GPT-4 model, but Truster improves the performance to 94.08 from the previous best refinement score of 93.10.

For StrategyQA, the trust module does not prove to be very helpful with a performance very similar to the refinement scores. This shows that it is difficult to train a Truster on fact-based datasets, as it is hard to rank two pieces of factual information without knowing the true facts.

Cost of fine-tuning LLMs vs. ART-based refinement Since the training samples are available for the GSM8K dataset, it is possible to fine-tune a LLaMA 70B model. Fine-tuning LLaMA 70B achieves 63.2% accuracy on GSM8K (Yuan et al., 2023), which is similar to what a trained 13B $\texttt{Asker}_{13B}$ and Truster can achieve with a pre-trained LLaMA 70B model, while incurring much lower training costs and computational requirements. Table 4 shows that training a 13B model as Truster is 10X cheaper than fine-tuning a 70B model, and even with two trained models as Asker and Truster, ART is still 5X cheaper. In addition, fine-tuning usually makes the model narrowly specialized to the trained dataset with reduced general in-context learning performance (Wang et al., 2022b), which won't happen with a pre-trained model deciding when to refine using our proposed framework ART.

Objective	Model Size	Flops	GPU Hours
Asker	7B	1.5×10^{17}	1
Truster	13B	3×10^{17}	4
FineTuning	70B	1.5×10^{18}	75

Table 4: Comparison of different compute requirements for training different sized LLaMA models on GSM8K with the objective of training a decision maker (Asker and Truster) vs. finetuning a model (FineTuning).

5 Ablation Studies

Importance of Asking Questions for Refinement We trained Asker to make only a binary decision of "Yes" or "No" to refine, without asking the relevant questions, and found that all versions of the LLaMA models always trusted the predictions and never decided to refine them. LLMs are often very bad at judging their own predictions and often prefer their own predictions (Kadavath et al., 2022), and our experiments observed a similar phenomenon. However, asking questions leads to a better refinement decision and a qualitative example is presented in Figure 4.

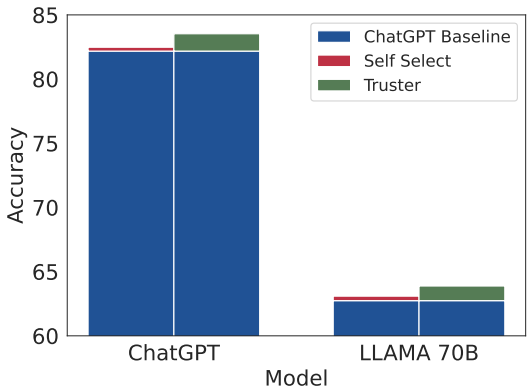

Figure 2: Comparison of the trained `Truster` with the self-selection version of the LLMs for GSM8K.

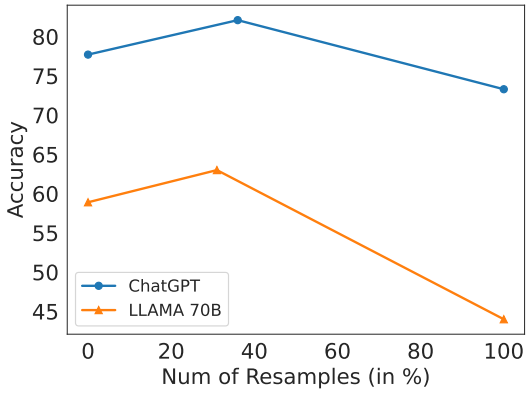

Figure 3: Number of resamples for refinement for ChatGPT and LLaMA 70B models on GSM8K. 0 means no resampling and 100 means resampling all the time.

Importance of `Truster` for selection We compared the performance of the selection module of the LLM (Self) vs. our trained `Truster` for the GSM8K dataset and observed that the trained `Truster` can better assess the errors made in the predictions and asks the model to revert to the previous generation more (about 50% more compared to self-selection); leading to superior performance (Figure 2).

When to refine? Assessing when to refine is an important component of the refinement pipeline, as always refining leads to worse results (Huang et al., 2023). Figure 3 supports the previous findings and shows that always refining can hurt the overall performance (100% refinement) and is worse than the initial prediction (0% refinement). The sweet spot is somewhere in the middle (about 30-35% refinement seems to work for both ChatGPT and LLaMA 70B models on the GSM8K dataset).

Can `Asker` be trained on its own output? Instead of training an `Asker` to ask questions on the output of the pre-trained LLM, can we train it on its own predictions? Much to our surprise, we find that `Asker` trained on its own data can make a better refinement decision than LLM's self-refinement. However, as expected, training on pre-trained model data proved to be more beneficial in deciding when to refine, due to a similar distribution of test and training samples, as shown in Table 5. However, for ChatGPT models, `Asker` trained on its own data performs similarly to that trained on LLaMA 70B models, as both data distributions are different from the ChatGPT test distribution (82.10 vs. 82.18 for the 13B model and 80.69 vs. 80.89 for the 7B model).

Initial Prediction	Refinement		
Accuracy	**Model**	**Data**	**Accuracy**
LLaMA 70B Pre-trained			
59.74	$\texttt{Asker}_{7B}$	7B	59.21
59.74	$\texttt{Asker}_{7B}$	70B	**61.33**
59.74	$\texttt{Asker}_{13B}$	13B	62.39
59.74	$\texttt{Asker}_{13B}$	70B	**62.74**
ChatGPT Turbo			
77.71	$\texttt{Asker}_{7B}$	7B	80.69
77.71	$\texttt{Asker}_{7B}$	70B	**80.89**
77.71	$\texttt{Asker}_{13B}$	13B	82.10
77.71	$\texttt{Asker}_{13B}$	70B	**82.18**

Table 5: Comparison of refinement results when `Asker` models are trained on their own data vs. LLaMA 70B pre-trained model data on the GSM8K dataset.

Can `ART` methodology be extended to SoTA models? We fine-tune MetaMath 7B and 13B (Yu et al., 2023) using the same training strategy as `Asker` models and compare the results in Table 6. MetaMath, due to its training on additional data and superior mathematical reasoning abilities, can evaluate the prediction better and leads to a better overall performance after refinement (62.31 vs. 61.33 for 7B and 64.06 vs. 62.74 for 13B variant). MetaMath models also perform better with fewer samples (about 27% fewer samples for 7B and 25% fewer for 13B), suggesting that superior mathematical reasoning can help to better evaluate predictions, leading to fewer uncertain samples for refinement. Since MetaMath was trained on over 250K samples with rejection sampling, it was not possible for us to run all experiments on this large dataset, and we stuck to LLaMA models for all of our experiments.

Question A customer's loyalty card at a store gives them rewards of $1 off their next purchase for every $20 they spend. Their last shopping trip, they spent $80. This shopping trip, they spent $43, used their rewards, and applied a coupon that took twice the amount of rewards off the price. How many dollars did the customer pay on this shopping trip?

Initial Prediction The customer spent $80 on their last shopping trip, so they earned 80/20 = «80/20=4»4 rewards. They used 2*4 = «2*4=8»8 rewards on this shopping trip. So the price of their purchase was $43 - $8 = $«43-8=35»35. The customer paid $35 on this shopping trip. The answer is 35. ✗

Self-Refinement Yes, the reasoning is correct. The customer spent $80 on their last shopping trip, earning them 80/20 = 4 rewards. They used 8 rewards on this shopping trip. Therefore, the price of their purchase was $43 - $8 = $35. ✗

Asker How much did the customer get in rewards? What was the coupons cost? How much did the customer pay on this shopping trip? All questions are not answered.

Refinement How much did the customer get in rewards? The customer had 80 / 20 = $«80/20=4»4 in rewards. What was the coupons cost? Their coupon was worth 4 * 2 = $«4*2=8»8. How much did the customer pay on this shopping trip? Thus they paid 43 - 4 - 8 = $«43-4-8=31»31 on this shopping trip. ✓

Figure 4: Qualitative example demonstrating the usefulness of asking questions to decide *when to refine* and then using those questions to refine the initial prediction by fixing the errors.

Initial pred **Accuracy**	**Asker**	Refinement **Acc (↑)**	**% samp (↓)**
59.74	LLaMA 7B	61.33	48
59.74	MetaMath 7B	**62.31**	35
59.74	LLaMA 13B	62.74	36
59.74	MetaMath 13B	**64.06**	27

Table 6: Comparison of LLaMA 7B and 13B refinement accuracy (Acc) with the state-of-the-art MetaMath 7B and 13B models (Yuan et al., 2023) and their sampling percentage (% samp) for refinement.

Entire ART pipeline in one go To test whether the entire ART pipeline of asking relevant questions, then deciding whether the questions are answered or not, and then refining can be learned in one go instead of individual models for each step, we train a LLaMA 13B and 70B model over the entire sequence (all-in-one-go). Figure 5 shows that all-in-one-go (green) performs worse than fine-tuning (orange) for the LLM, demonstrating that generating the entire sequence is a more challenging task for the LLM than individual components.

6 Key Findings

From the experiments, we observe the following:

- **ART allows smaller models to make refinement decisions superior to LLM self-refinement**: Smaller models trained to make a refinement decision can outperform a much larger model in *self-refinement* style (Table 2).
- **Ask questions before refinement** Asking questions is an effective way to verify the quality of the generations and allows the models to make better refinement decisions.

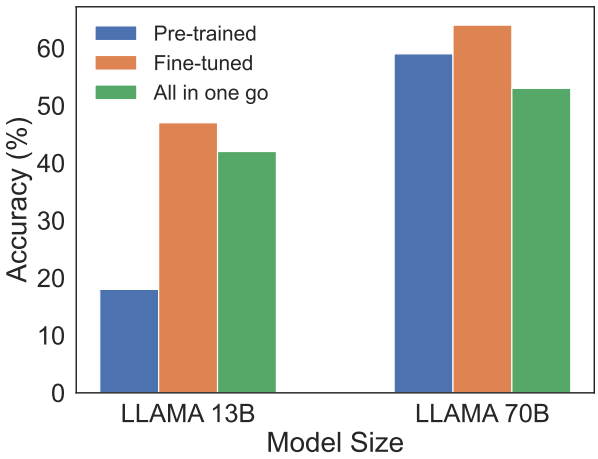

Figure 5: Comparison of the all-in-one approach to fine-tuning the LLMs on the GSM8K dataset.

- **Smaller models' refinement decisions are a cost-effective alternative to fine-tuning LLMs** The refinement decision of smaller models combined with a pre-trained LLM performs similarly to a larger model when fine-tuned. This saves a lot of computation required to fine-tune a larger model (Table 4) and preserves downstream performance on other tasks.
- **Expert models can make better judgments about refinement** Larger models ($\texttt{Asker}_{13B}$ performance is better than $\texttt{Asker}_{7B}$ in all cases) show that better models can make more informed decisions about when to refine. Table 6 shows that MetaMath trained models outperform LLaMA models of similar size.
- **Trained Truster can rank decisions better**

A trained smaller Truster model can rank the results better than the self-selection version of LLMs, as shown in Figure 2.

7 Conclusion

In this work, we propose a refinement strategy called ART: Ask, Refine, and Trust, which allows smaller models to make refinement decisions for LLMs and determine whether these refinements are reliable. We empirically demonstrate the effectiveness of our approach on two reasoning tasks, mathematical word problems and question answering. Our results show that smaller models, even up to 10X smaller, can outperform larger models in making refinement decisions.

Limitations

In this work, we trained a Asker to make a refinement decision by asking questions to verify the predictions. We used the training data available for the GSM8K and StrategyQA datasets. However, for many tasks, training data may not be available. In such cases, LLMs can be used to generate data and in many cases it performs similarly to the ground truth data (Magister et al., 2023). However, we have not tested this with ART due to the availability of the training dataset. In addition, for StrategyQA, we used the available facts to support the model decision when refining the predictions. These facts were available in the dataset, but in the real world can be extracted with the help of some tools or from some databases. We did not test this approach in our work and leave it for future work.

References

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.

Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. 2005. Learning to rank using gradient descent. In *Proceedings of the 22nd International Conference on Machine Learning*, ICML '05, page 89–96, New York, NY, USA. Association for Computing Machinery.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems.

Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. 2023. Chain-of-verification reduces hallucination in large language models. *ArXiv*, abs/2309.11495.

Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. 2021. Did Aristotle Use a Laptop? A Question Answering Benchmark with Implicit Reasoning Strategies. *Transactions of the Association for Computational Linguistics (TACL)*.

Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander J. Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. *ArXiv*, abs/2305.02301.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2023. Large language models cannot self-correct reasoning yet. *ArXiv*, abs/2310.01798.

Saurav Kadavath, Tom Conerly, Amanda Askell, T. J. Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zachary Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, John Kernion, Shauna Kravec, Liane Lovitt, Kamal

Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom B. Brown, Jack Clark, Nicholas Joseph, Benjamin Mann, Sam McCandlish, Christopher Olah, and Jared Kaplan. 2022. Language models (mostly) know what they know. *ArXiv*, abs/2207.05221.

Takeshi Kojima, Shixiang (Shane) Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems*, volume 35, page 22199–22213. Curran Associates, Inc.

Kalpesh Krishna, Yapei Chang, John Wieting, and Mohit Iyyer. 2022. Rankgen: Improving text generation with large ranking models. *ArXiv*, abs/2205.09726.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Venkatesh Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. 2022. Solving quantitative reasoning problems with language models. *ArXiv*, abs/2206.14858.

QING LYU, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. 2023. Faithful chain-of-thought reasoning. *ArXiv*, abs/2301.13379.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems, 2023*.

Lucie Charlotte Magister, Jonathan Mallinson, Jakub Adamek, Eric Malmi, and Aliaksei Severyn. 2023. Teaching small language models to reason. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1773–1781, Toronto, Canada. Association for Computational Linguistics.

Sewon Min, Victor Zhong, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2019. Multi-hop reading comprehension through question decomposition and rescoring. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 6097–6109, Florence, Italy. Association for Computational Linguistics.

OpenAI. 2023. Gpt-4 technical report.

Debjit Paul, Mete Ismayilzada, Maxime Peyrard, Beatriz Borges, Antoine Bosselut, Robert West, and Boi Faltings. 2023. Refiner: Reasoning feedback on intermediate representations. *ArXiv*, abs/2304.01904.

Ansh Radhakrishnan, Karina Nguyen, Anna Chen, Carol Chen, Carson E. Denison, Danny Hernandez, Esin Durmus, Evan Hubinger, John Kernion, Kamil.e Lukovsiut.e, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Sam McCandlish, Sheer El Showk, Tamera Lanham, Tim Maxwell, Venkat Chandrasekaran, Zac Hatfield-Dodds, Jared Kaplan, Janina Brauner, Sam Bowman, and Ethan Perez. 2023. Question decomposition improves the faithfulness of model-generated reasoning. *ArXiv*, abs/2307.11768.

Kumar Shridhar, Harsh Jhamtani, Hao Fang, Benjamin Van Durme, Jason Eisner, and Patrick Xia. 2023a. Screws: A modular framework for reasoning with revisions. *ArXiv*, abs/2309.13075.

Kumar Shridhar, Jakub Macina, Mennatallah El-Assady, Tanmay Sinha, Manu Kapur, and Mrinmaya Sachan. 2022. Automatic generation of socratic subquestions for teaching math word problems. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.

Kumar Shridhar, Alessandro Stolfo, and Mrinmaya Sachan. 2023b. Distilling reasoning capabilities into smaller language models. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 7059–7073, Toronto, Canada. Association for Computational Linguistics.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Daniel M. Bikel, Lukas Blecher, Cristian Cantón Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony S. Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel M. Kloumann, A. V. Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, R. Subramanian, Xia Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zhengxu Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open foundation and fine-tuned chat models. *ArXiv*, abs/2307.09288.

Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Alexander M. Rush, and Thomas Wolf. 2023. The alignment handbook. `https://github.com/huggingface/alignment-handbook`.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Huai hsin Chi, and Denny Zhou. 2022a. Self-Consistency improves chain of thought reasoning in language models. *ArXiv*, abs/2203.11171.

Yihan Wang, Si Si, Daliang Li, Michal Lukasik, Felix X. Yu, Cho-Jui Hsieh, Inderjit S. Dhillon, and Surinder Kumar. 2022b. Two-stage llm fine-tuning with less specialization and more generalization.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed H. Chi, Quoc V Le, and Denny Zhou. 2022. Chain of thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*.

Sean Welleck, Ximing Lu, Peter West, Faeze Brahman, Tianxiao Shen, Daniel Khashabi, and Yejin Choi. Generating sequences by learning to self-correct. *In The Eleventh International Conference on Learning Representations, 2023*.

Ori Yoran, Tomer Wolfson, Ben Bogin, Uri Katz, Daniel Deutch, and Jonathan Berant. 2023. Answering questions by meta-reasoning over multiple chains of thought. *ArXiv*, abs/2304.13007.

Long Long Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zheng Li, Adrian Weller, and Weiyang Liu. 2023. Metamath: Bootstrap your own mathematical questions for large language models. *ArXiv*, abs/2309.12284.

Zheng Yuan, Hongyi Yuan, Cheng Li, Guanting Dong, Chuanqi Tan, and Chang Zhou. 2023. Scaling relationship on learning mathematical reasoning with large language models. *ArXiv*, abs/2308.01825.

Zhuosheng Zhang, Aston Zhang, Mu Li, and Alexander J. Smola. 2022. Automatic chain of thought prompting in large language models. *ArXiv*, abs/2210.03493.

Chuanyang Zheng, Zhengying Liu, Enze Xie, Zhenguo Li, and Yu Li. 2023. Progressive-hint prompting improves reasoning in large language models.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. 2023. Least-to-most prompting enables complex reasoning in large language models. *In The Eleventh International Conference on Learning Representations, 2023*.

Question A customer's loyalty card at a store gives them rewards of $1 off their next purchase for every $20 they spend. Their last shopping trip, they spent $80. This shopping trip, they spent $43, used their rewards, and applied a coupon that took twice the amount of rewards off the price. How many dollars did the customer pay on this shopping trip?

Initial Prediction The customer spent $80 on their last shopping trip, so they earned 80/20 = «80/20=4»4 rewards. They used 2*4 = «2*4=8»8 rewards on this shopping trip. So the price of their purchase was $43 - $8 = $«43-8=35»35. The customer paid $35 on this shopping trip. The answer is 35. ✗

`Ask` How much did the customer get in rewards? What was the coupons cost? How much did the customer pay on this shopping trip? All questions are not answered.

`Refine` *How much did the customer get in rewards?* The customer had 80 / 20 = $«80/20=4»4 in rewards. *What was the coupons cost?* Their coupon was worth 4 * 2 = $«4*2=8»8. *How much did the customer pay on this shopping trip?* Thus they paid 43 - 4 - 8 = $«43-4-8=31»31 on this shopping trip. ✓

`Trust` The customer had 80 / 20 = $«80/20=4»4 in rewards. Their coupon was worth 4 * 2 = $«4*2=8»8. Thus they paid 43 - 4 - 8 = $«43-4-8=31»31 on this shopping trip. ✓

Question The cell-phone recharges at the rate of 1 percentage-point of charge per 3 minutes. Now, the phone is at 60% charged. How long will it take to fully charge, in hours?

Initial Prediction The phone needs 100-60 = «100-60=40»40% more charge to be fully charged. Since the phone recharges at the rate of 1 percentage-point of charge per 3 minutes, it will take 40*3 = «40*3=120»120 minutes to fully charge. Therefore, it will take 120/60 = «120/60=2»2 hours to fully charge. The answer is 2. ✓

`Ask` How many percentage points is left to be charged? How many minutes will it take to fully charge? How long will it take to fully charge, in hours? All questions are answered.

Figure 6: End to end flow of our proposed objective of `ART: Ask, Refine, and Trust` if `Asker` decides to refine (top) vs no refinement needed (bottom).

Question Claire makes a 3 egg omelet every morning for breakfast. How many dozens of eggs will she eat in 4 weeks?

Subquestions How many eggs does Claire eat in a week? How many eggs will Claire eat in 4 weeks? How many dozens of eggs will Claire eat in 4 weeks?

Answer She eats 3 eggs every day and there are 7 days in a week so she eats 3*7 = «3*7=21»21 eggs a week. After 4 weeks she will have eaten 4*21 = «4*21=84»84 eggs. There are 12 eggs in 1 dozen and she'll eat 84 eggs so that's 84/12 = «84/12=7»7 dozen eggs.

Question Can a greyhound walk on two legs?

Subquestions What type of animal is a greyhound? Does #1 walk on two legs?

Facts Greyhounds are dogs. Dogs walk on four legs.

Answer False

Figure 7: Example of a GSM8K data sample (top) and StrategyQA data sample (bottom).